

Thanks for purchasing Nav3D!

For all questions, please email: support@softbrix.studio

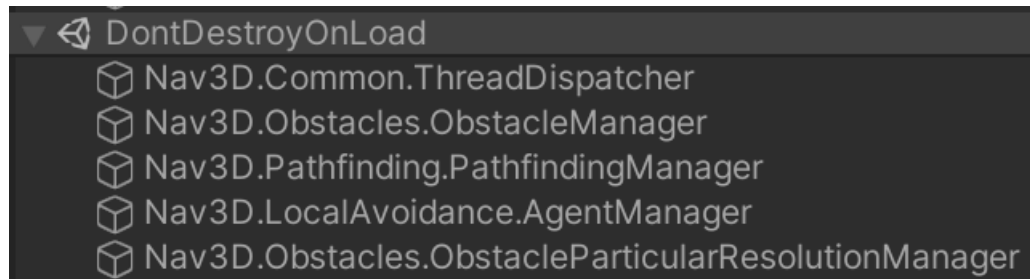
Look for much more information on the [asset website](#)!

[Video tutorials](#)!

Thanks for purchasing Nav3D!	1
Getting started	3
Nav3DInitializer	3
Nav3DManager	4
Dealing with obstacles	5
Nav3DObstacle	5
Nav3DObstacleLoader	7
Deeper dive into obstacles	10
Operations with obstacles	10
Obstacle combinations	11
Particular resolution regions	12
Nav3DParticularResolutionRegion	13
Nav3DParticularResolutionRegion : Some points	14
Path object	15
Nav3DPathfindingManager	15
Pathfinding	15
PathfindingError	16
Path:Properties	17
Completing dealing with the Path Object	17
Usage example	17
Agents	19
Nav3DAgentDescription	19
Randomly generated parameters	27
Creating and configuring an agent description from code	28
Nav3DAgent	28
Nav3DAgent : Debug	30
Debug drawing	30
Agent log	34

Getting started

- All asset classes you need to work with are contained in the Nav3D.API namespace.
- In the playmode, Nav3D will create several MonoBehaviour objects on the scene that it needs for internal use.



These GameObjects are created with the DontDestroyOnLoad flag. Saving managers when loading another scene can be useful if, for example, you use a separate loading scene in which you initialize Nav3D.

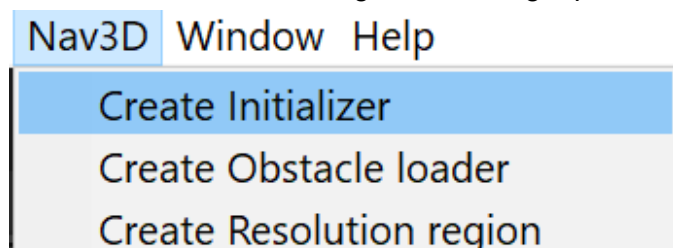
- Movement of agents (Nav3DAgent) executes inside the FixedUpdate event.
- All time-consuming computational operations are performed on the CPU outside the main thread.
- All callbacks provided in the Nav3D API are executed in the MainThread.

Nav3DInitializer

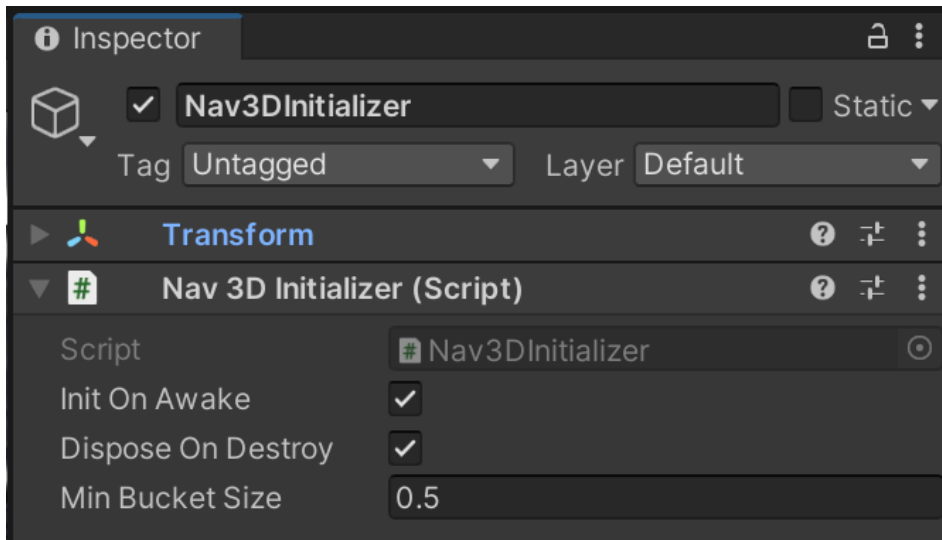
To use Nav3D in playmode, you need to initialize it.

The Nav3DInitializer component is designed for this.

Create it on the scene using the following top menu button:



The following game object will appear on the scene:



It contains the following settings:

- **Init On Awake** - Leave this flag enabled to initialize Nav3D in the playmode on Awake event invocation.
- **Dispose On Destroy** - Leave this flag enabled to force Nav3D to clean up all of its internal entities when destroying scene. If disabled, Nav3D and its entities will continue to work when a new scene is loaded.
- **Min Bucket Size** - the minimum size of the navigation graph buckets. Must be a positive non-zero number. Here we will explain in a little more detail. Essentially, this parameter allows you to customize the level of detail of the navigation graph. The smaller the parameter value, the higher the detail of the graph.

All game agents that will search for a path using Nav3D have their own size, determined by the radius that you give them when creating. (We will explain this a little later). The minimum cell size must be equal to the maximum radius of the agents, this will guarantee that the agent can go along any path in the graph. If there are agents on the scene whose radius exceeds the minimum bucket size of the navigation graph, then situations are possible when the agents collide obstacles on the stage. Future versions of Nav3D will implement agent size-dependent pathfinding.

**If the Init On Awake and Dispose On Destroy options are disabled, it will be your responsibility to initialize and dispose Nav3D resources. For initialization you will need to call the Nav3DInitializer.Init() method, for disposal - Nav3DInitializer.Utilize(). The value of the Min Bucket Size parameter can also be set via code; to do this, use the MinBucketSize property.*

Nav3DManager

Nav3DManager is a helper static class. Can be useful for checking whether Nav3D has initialized and/or performing certain actions on the initialization.

Contains:

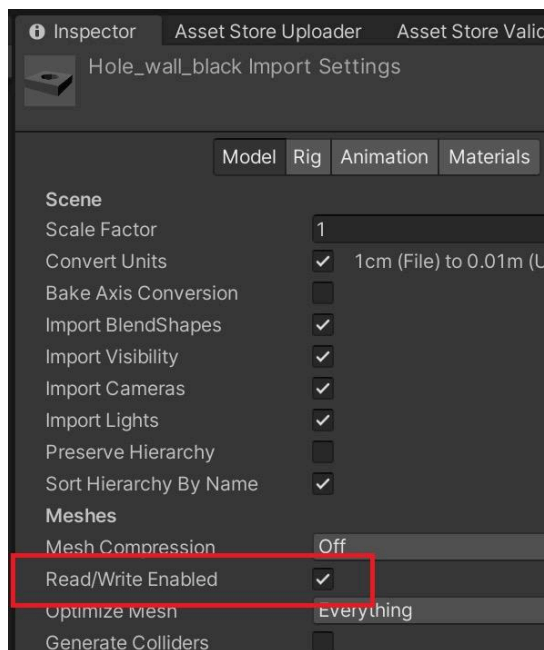
- A *bool Init* property that indicates whether Nav3D has already been initialized.
- Event *event Action OnNav3DInit*. It fires immediately after Nav3D is initialized. By subscribing to it you will be able to perform any actions with Nav3D initialization. When you subscribe to an event, your delegate will work instantly, if initialization has already been done previously.

If you try to use public methods from Nav3D.API namespace before Nav3D is initialized, a Nav3DManagerNotInitializedException will be thrown. Any actions with Nav3D entities may be performed only after its initialization.

Dealing with obstacles

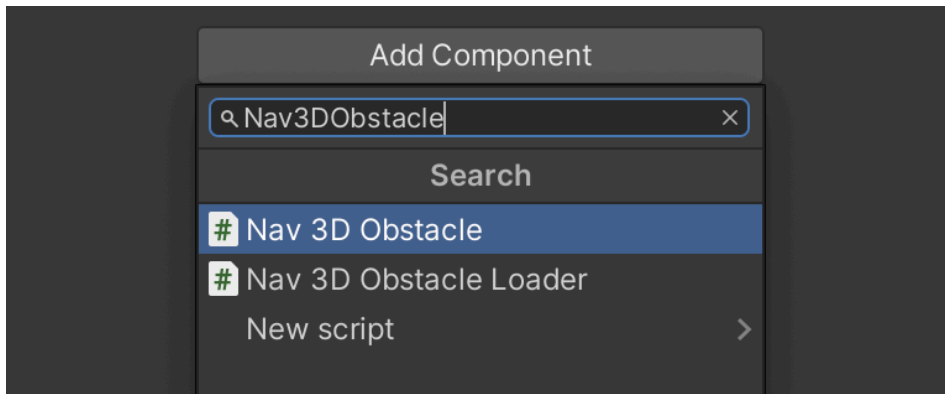
Nav3D allows you to use game objects in the scene as obstacles with a MeshFilter component with a Mesh assigned, or with a Terrain component.

**For obstacles using MeshFilter, make sure the mesh import settings “Read/Write Enabled” flag is set to true.*

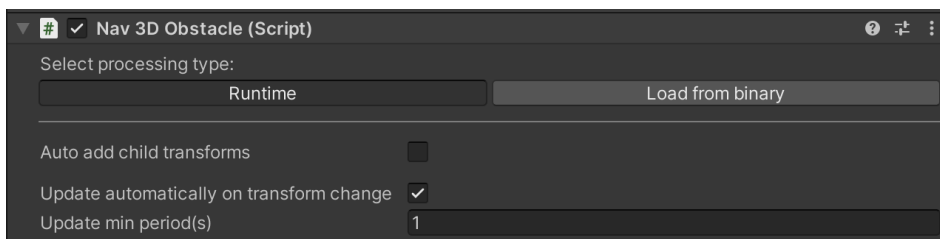


Nav3DObstacle

To manage obstacles on the scene, the Nav3DObstacle component is designed. Attach it to a game object that you want to be an obstacle in the scene.



You will see the component inspector:



It has the following settings:

- Selecting an obstacle processing mode (obstacles processing must be performed after Nav3D is initialized, so that obstacles are taken into account when pathfinding performs to agents can avoid them while moving):
 - Runtime - the obstacle will be processed directly during playmode after Nav3D is initialized. This can be useful if an obstacle is generated during playmode.
 - Load from binary - the obstacle will be loaded from a pre-baked binary file. In many cases, the configuration of a game scene is known in advance, even before the scene is loaded and Nav3D is initialized. In such cases, it always makes sense to bake scene obstacles in editor mode, and load them from a binary file at the start of the scene. Loading from a binary file is always much faster than processing an obstacle directly in runtime.
- Auto add child transforms - if enabled, then information about the MeshFilter and Terrain components will be collected for all children of the transform hierarchy. And all children who have these components will be taken into account when processing the obstacle.

**If you want any child element of an obstacle containing a MeshFilter or Terrain not to be taken into account when constructing the navigation graph, then attach the Nav3DObstacleIgnoreTag component to it.*

- Update automatically on transform change - if enabled, then if the position, rotation or scale of the transform component of an obstacle changes, it will cause it to recalculate the obstacle's navigation graph. (The obstacle will first be removed from the obstacle store, then re-processed and added to it.)

This function is available for use only for obstacles with the selected Runtime processing mode.

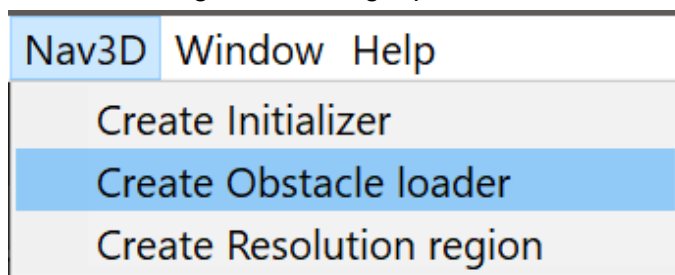
**Since adding an obstacle is always a time-consuming process, updating the obstacle too often can have a negative impact on performance. We recommend using this function for small, isolated obstacles.*

- Update min period - how often the obstacle should be updated (in seconds) when the "Update automatically on transform change" option is enabled.

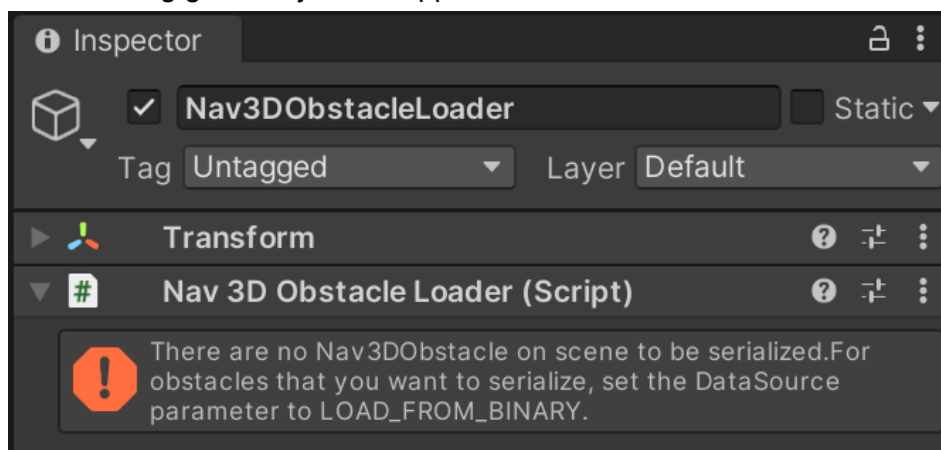
For runtime obstacles, it is possible to remove/re-add them to the storage directly during the playmode. Addition occurs when `OnEnable()` is invoked, removal occurs when `OnDisable()` is invoked. Accordingly, any runtime obstacle created on the scene during the playmode will be processed and added to the storage. If you delete or disable an obstacle game object, it will be removed from storage. Re-enabling an obstacle will re-add it to storage.

Nav3DObstacleLoader

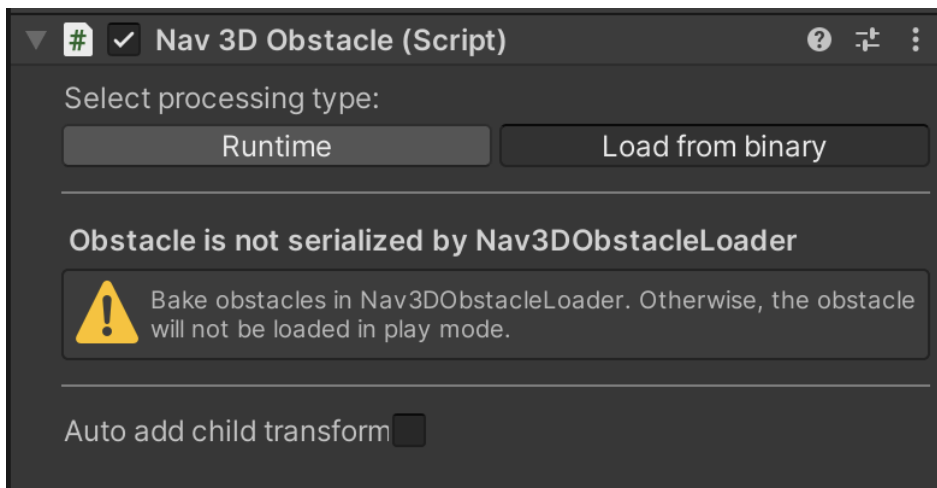
To use the possibility of pre-baking obstacles on the scene in editor mode and then loading them from a binary file, the Nav3DObstacleLoader component is used. You can create it on the scene using the following top menu button:



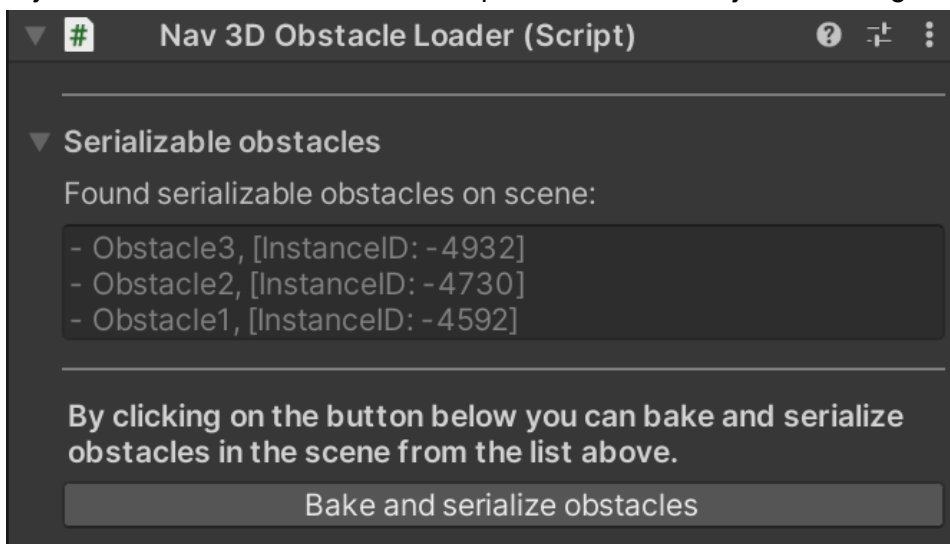
The following game object will appear on the scene:



For all obstacles in the scene that you want to bake, select the "Load from binary" processing mode.

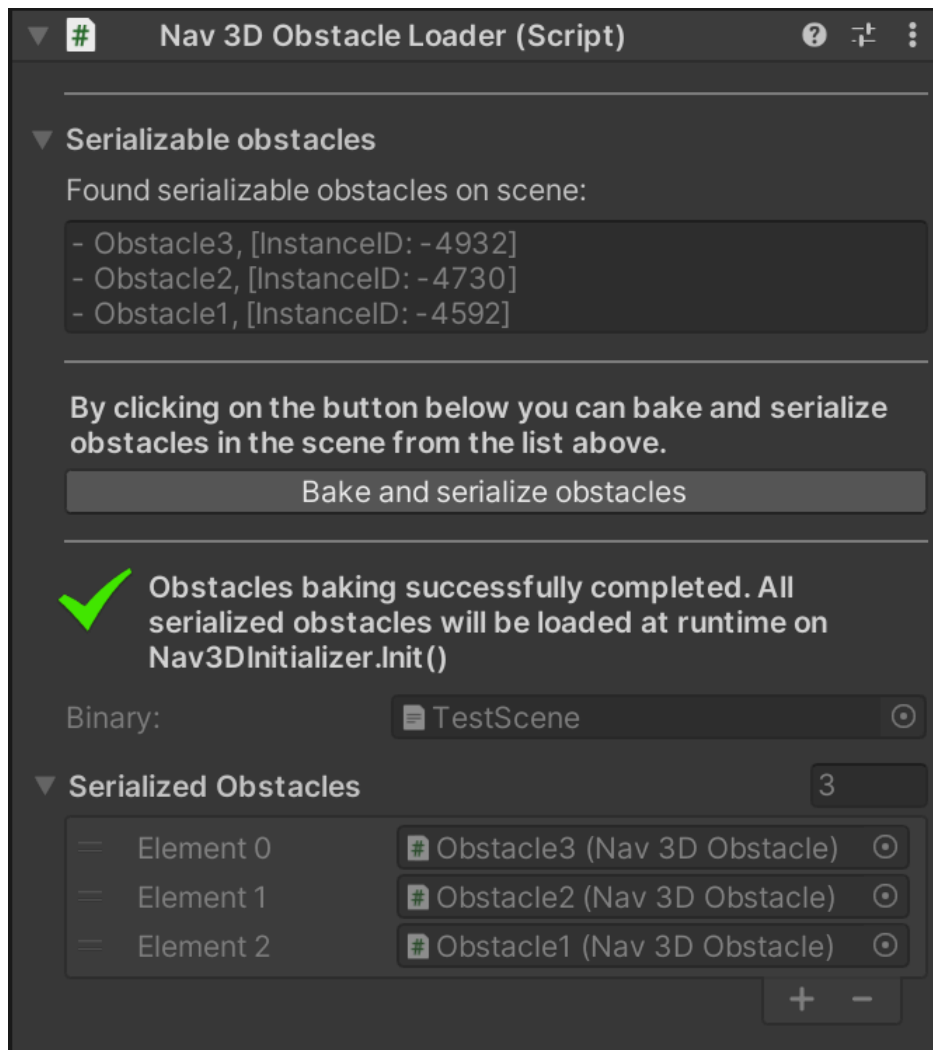


Next, in the Nav3DObstacleLoader component inspector, you can see a list of all game objects with the Nav3DObstacle component that are subject to baking.

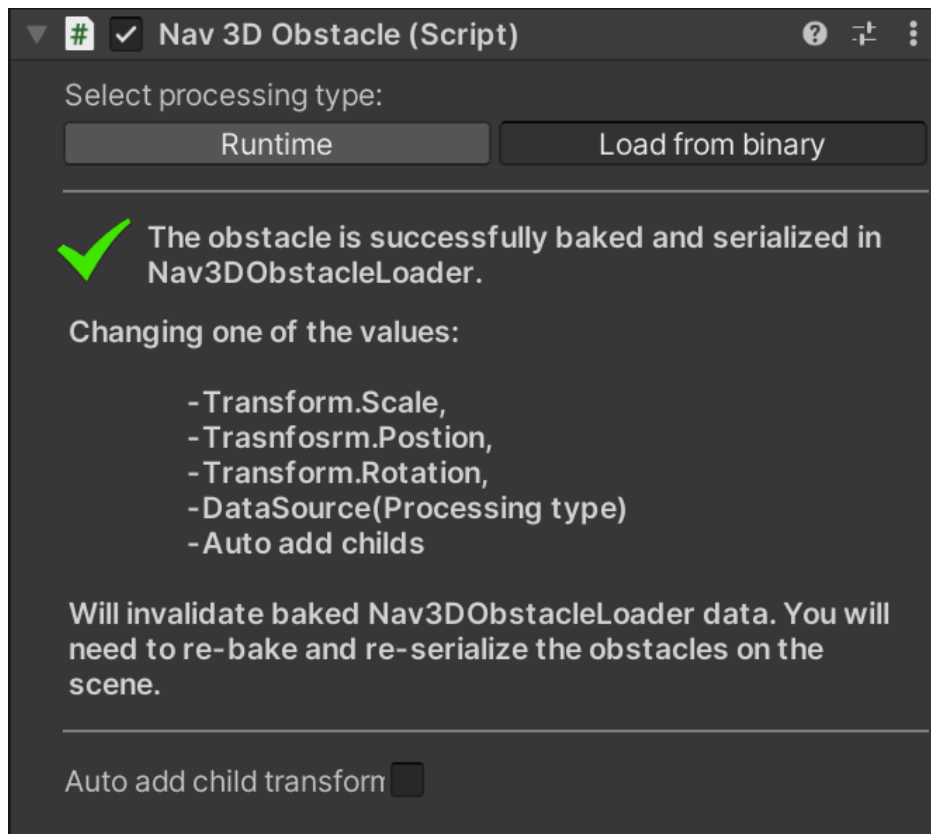


Click the “Bake and serialize obstacles” button and select a folder in the project to save a binary file with baked data about obstacles on the scene.

After successful baking, the Nav3DObstacleLoader component inspector will look like this:



The inspector of successfully serialized Nav3DObstacle will look like this:



Now in game mode, when Nav3D is initialized, the navigation graphs of all baked obstacles on the scene will be loaded from a binary file.

Deeper dive into obstacles

- As already described above, in order for an obstacle to be taken into account during pathfinding, you need to attach a Nav3DObstacle component to its game object.
- Nav3D has an obstacle storage that is used during pathfinding. It stores navigation graphs of obstacles on the scene.
- All operations with obstacles (adding or removing) are placed in an execution queue and performed sequentially in a separate thread.
- Obstacle processing means constructing a passability graph (also known as a navigation graph) for each obstacle, or for a group of obstacles, and then adding this graph to the obstacle storage.
Navigation graphs are used during pathfinding on a scene.
The result of processing one obstacle or a group of obstacles can be one or more navigation graphs that do not intersect each other in space.

Operations with obstacles

Regardless of the obstacle processing mode, the processing procedure consists of several stages:

1. **Collecting information about the geometry of the obstacle for each Nav3DObstacle.**

At this stage, a list of all geometries that form an obstacle is obtained, then they are combined (clustered).

Let's say there are several children in the transform hierarchy of the obstacle, each of which has a MeshFilter with a Mesh assigned. Accordingly, in this case, several lists of triangles that make up the meshes will be obtained. Next, for each list of triangles, the enclosing volume will be defined in the form of Bounds. Those lists whose Bounds intersect will be merged. We will call each such individual list an "obstacle volume." The result is one or more obstacle volumes. This will be done for every Nav3DObstacle processed in the scene.

2. **Clustering with already processed obstacles.**

It is checked whether the volumes of the newly processing obstacle intersect with the volumes of obstacles already in the storage. If yes, then clustering occurs with obstacles already added. The result is a new list of obstacle volumes.

3. **Removing obsolete volumes of stored obstacles from storage.**

Those volumes that intersected with the volumes of the newly processing obstacle and that were included in the new (clustered) volumes are considered obsolete.

4. Next, **navigation graphs are constructed for new volumes.**

These volumes are then added to the obstacle storage.

Removing an obstacle from a storage consists of the following steps:

1. From the storage, a list of all volumes containing triangles of the obstacle being removed is obtained (each volume can contain triangles of several obstacles).
2. For each volume, the triangles of the obstacle to be removed are removed from the resulting list.
3. Further, only volumes in which triangles still remain after removal are considered. For these volumes, the embracing Bounds are recalculated.
4. All volumes from the original list are removed from the obstacle storage.
5. All remaining volumes with recalculated Bounds are clustered and added back to the obstacle storage.

Obstacle combinations

We will call obstacles with the selected processing mode "Runtime" as runtime obstacles, and those with the "Load from binary" mode as static obstacles.

If you create a runtime obstacle during game mode, it will be processed and added to the storage (when OnEnable() is triggered). If you disable an already processed runtime obstacle on the scene, it will be removed from the storage (when OnDisable() is triggered).

Static obstacles are meant to be static in the sense that they cannot be added or removed in playmode. They will be present in the scene from the moment they are loaded from the file until Nav3D is deinitialized.

Since the above operations of adding and removing for a specific obstacle can affect the volumes of other obstacles on the scene, a conflict arises between static and runtime obstacles if their volumes intersect in space.

So we decided to introduce a few restrictions:

- At the time of initialization, all static obstacles are first loaded and added, only then runtime obstacles are processed and added (if there are any on the scene).
- When processing and adding a runtime obstacle, its volume must not intersect the volumes of static obstacles, otherwise an `IntersectStaticObstacleException` will be thrown.
- As mentioned above, adding several runtime obstacles can lead to the fact that a combined volume will be formed for them that is larger than each of their volumes separately. Your task is to ensure that the possible combined volume does not intersect the volumes of static obstacles. To check for such a potential collision there is a method `Nav3DManager.BoundsCrossStaticObstacles(Bounds _Bounds)`.

To avoid such situations, we suggest that you completely avoid using static and runtime obstacles together on the same scene.

If you still decide to use both types, we recommend creating runtime obstacles far from static obstacles in order to minimize the likelihood of volumes intersecting.

Particular resolution regions

It is possible to define space regions in which a particular minimum bucket size of the navigation graph can be specified.

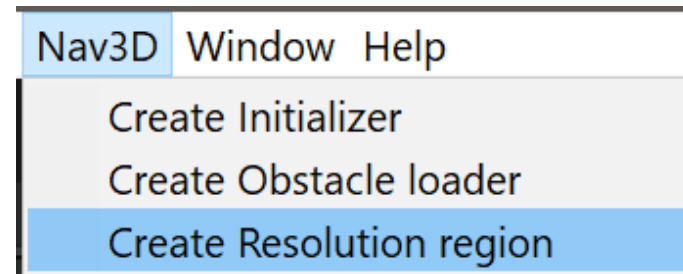
Let's imagine a situation where a suitable minimum bucket size of the search graph (resolution) is 1. However, there are several tight areas in your game scene where a resolution of 1 does not give sufficient detail to the navigation graph. In such a situation, you can of course reduce the minimum size of the search graph buckets for the entire space of your scene, but this will lead to an increase in the processing time for obstacles, as well as an increase in the average pathfinding time on the scene.

The best way out in such a situation would be to reduce the size of the buckets only in the area where you want to increase the detail of the search graph.

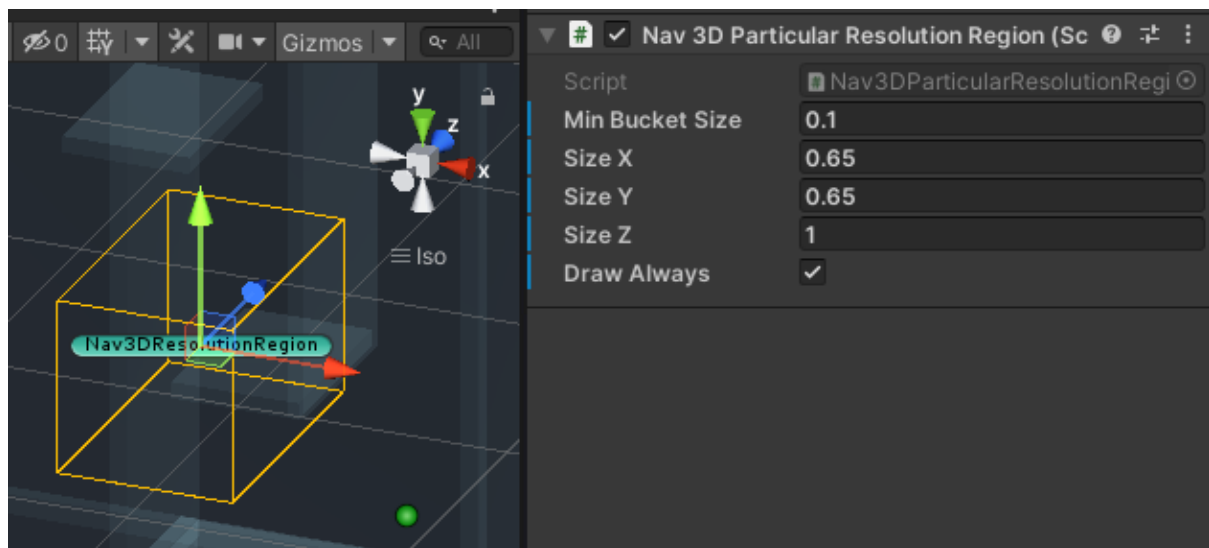
To do this, we implemented the `Nav3DParticularResolutionRegion` object, which allows you to specify a region in space in which a specific minimum bucket size of the navigation graph will be used when processing an obstacle.

Nav3DParticularResolutionRegion

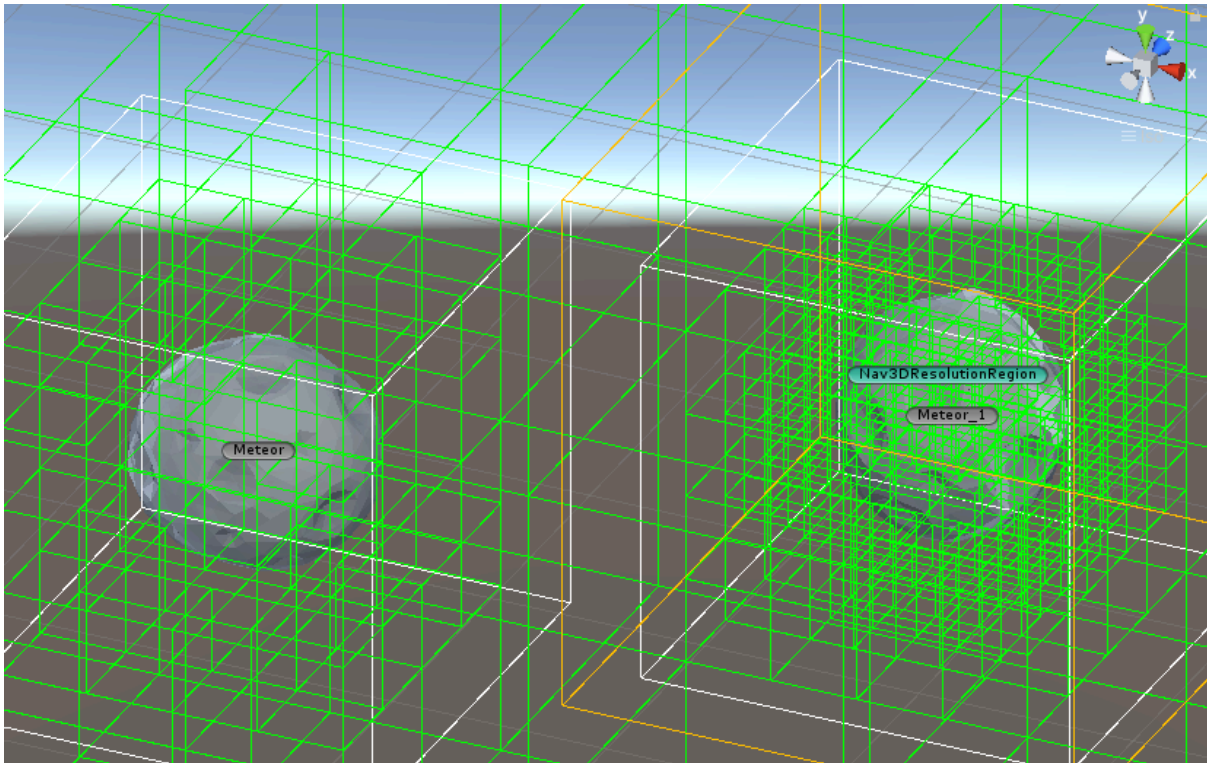
To create graph particular resolution region, use the following top menu button:



Set the required position on the scene for it, then in the inspector of the Nav3DParticularResolutionRegion component, set the region sizes, and also specify the desired minimum size of the navigation graph buckets. To draw the region, turn on Gizmos in the scene window.



Below is a navigation graph for two identical obstacles, one of which is located in a region with a minimum bucket size of the graph = 0.2. The second one is outside the region and the minimum bucket size of the pathfinding graph on the whole scene is set as = 0.4.



The obstacle on the right obviously has a more detailed navigation graph, since it is located in a region with a smaller minimum bucket size.

Nav3DParticularResolutionRegion : Some points

1. The use of the resolution regions is intended to solve problems related to the insufficiency of the graph detailing in tight areas of space, such as, for example, narrow tunnels, small holes through which it must be possible to find a path. But also in such regions, you can set the size of the minimum bucket larger than the bucket size set on the entire scene. This can be useful when, on the contrary, you want to avoid over-detailing the search graph for some obstacles. Avoiding excessively high resolution of the navigation graph will have a beneficial effect on overall pathfinding performance.
2. An octree is a hierarchical structure in which the sizes of all buckets are always a multiple of each other's size and from level to level the size changes with a multiplier of two (each next level contains buckets two times smaller than the previous one). So you can only specify the desired minimum bucket size. In fact, the actual minimum bucket size will be calculated relative to the size specified during Nav3D initialization. If, for example, at initialization, the minimum size was specified as 0.4, and you specify the desired minimum size as 0.3, then the size 0.2 will actually be chosen, since it should be a level lower and obtained from 0.4 divided by 2. If you choose the desired size as 0.7, then the actual size will be 0.8 since it must be a level higher and is multiplied by 2. Desired value 0.9 leads in result to 1.6 value. And so on.
3. If the obstacle is located inside the intersection of several regions with different minimum bucket sizes, the smallest value from the region values will be selected as the minimum bucket size during processing.

4. We recommend using regions as static objects that are on the scene when all obstacles are processed. The operation of adding/removing a region during runtime will entail re-processing those runtime obstacles that are intersected by the region. Static obstacles will not react in any way to removing/adding a region. Initialization and deinitialization of a region occurs through the `OnEnable()` and, accordingly, `DoDisable()` events in `Nav3DParticularResolutionRegion`. So to initialize a region, you can create its prefab on the stage, to deinitialize it, you can make its `GameObject` inactive, or destroy it.

Path object

The `Path` class is designed to work with paths in space.

In general, objects of this type are used by agents (`Nav3DAgent`) when performing a path search. But you can also use `Path` to find the path for your own purposes.

Of course, working with `Path` objects is only possible after `Nav3D` has been initialized. Read more in “`Nav3D Initialization`” section.

Nav3DPathfindingManager

This class contains tools for working with pathfinding in the scene.

To achieve the required level of performance when using `Nav3D`, the following properties can be useful:

- `public static int CurrentPathfindingTasksCount` - shows how many parallel pathfinding tasks are currently active;
- `public static int MaxPathfindingTasks` - allows you to set a limit on the maximum number of pathfinding tasks running at the same time. By default, this property is set to `Environment.ProcessorCount - 1`

There is also a public static `Path PrefetchPath(Vector3 _PointA, Vector3 _PointB)` method to create a `Path` instance. Its application is discussed below.

Pathfinding

To find a path between points A and B, you must:

- 1) Get an object of type `Path` by calling the `Nav3DPathfindingManager.PrefetchPath(Vector3 _PointA, Vector3 _PointB)` method. This method will only create an instance of `Path` and register it with `Nav3D`, but no pathfinding will be performed.
- 2) On the created `Path` object, call the method

```

UpdatePath(
Vector3? _Start=null,
    Vector3? _Goal=null,
    int? _Timeout = null,
    bool? _Smooth = null,
    Action_OnSuccess = null,
    Action<PathfindingError> _OnFail = null
)

```

Values of overloaded parameters:

- Vector3? _Start - starting point of the path. The parameter must be passed if you want to change the starting point of the path.
- Vector3? _Goal - the end point of the path. The parameter must be passed if you want to change the end point of the path.
- int? _Timeout - maximum path search time. In case the path search takes longer, the _OnFail callback will be executed if it is not null.
- bool? _Smooth = null - whether it is necessary to smooth the found path.
- Action _OnSuccess = null - callback that will be called after successful path finding
- Action<PathfindingError> _OnFail is a callback that will be called if the path could not be found for any reason.

PathfindingError

The class whose instance is returned to the _OnFail callback of the Path.UpdatePath() method.

Contains two properties:

- public PathfindingResultCode Reason - The reason why the pathfinding failed.
Possible values:

Value	Description
SUCCEEDED	Pathfinding finished successfully.
PATH_DOES_NOT_EXIST	There is no path between points. This is possible if at least one of the neighborhood points is surrounded by all sides.
TIMEOUT	The pathfinding took longer than allowed and was aborted.
CANCELED	Pathfinding was canceled by the user or Nav3D internal logic.
START_POINT_INSIDE_OBSTACLE	Pathfinding has been canceled because the start point of the path is inside an obstacle.

GOAL_POINT_INSIDE_OBSTACLE	Pathfinding has been canceled because the goal point of the path is inside an obstacle.
UNKNOWN	Internal error.

- public string Msg - message with more detailed information about the error.

Path:Properties

- public Vector3[] Trajectory { get; } - the found path. Use this property to get the last found path. The value is true if the UpdatePath method was called and completed successfully, and no other properties of the Path object have changed since then. Use the IsValid property to verify that a value is valid.
- public Vector3[] TrajectoryOriginal {get;} - original found path.
- public Vector3[] TrajectoryOptimized {get;} - optimized found path.
- public Vector3[] TrajectorySmoothed { get; } - smoothed found path. The value will be different from TrajectoryOptimized if the Smooth property is set to true.
- public Bounds Bounds { get; } - the amount of space occupied by the found path.
- public Vector3 Start { get; set; } - the starting point of the path.
- public Vector3 Goal { get; set; } - the end point of the path.
- public int SmoothRatio { get; set; } - the number of smoothing passes in relation to the minimum cell size of the search graph, by default it is set to 3. We do not recommend increasing this value unnecessarily, as this will increase the path search time if smoothing is enabled (Smooth == true property).
- public int Timeout { get; set; } - maximum path search time. If the path search takes longer, the Action<PathfindingError> _OnFail callback will be executed if it was passed to UpdatePath().
- public bool Smooth { get; set; } - whether it is necessary to smooth the path.
- public bool IsValid { get; } - whether the current value of the Trajectory is not null and whether it is true for the current values of the remaining properties. For example, if you called UpdatePath(), and after its successful completion you changed the value of the Start property, then the path will no longer be up-to-date and IsValid will be equal to false.
- public bool IsPathUpdating { get; } - whether the Path instance is in the process of pathfinding.

Completing dealing with the Path Object

After you've finished dealing with the Path object, you need to call the Path.Dispose() method. This is necessary to remove the path from the obstacle storage. Otherwise, when adding / updating / removing any obstacle, the path will be checked for the need to update, which will take up computing resources.

Usage example

The following is an example of using the Path object to find the path from A to B.

```

using Nav3D.API;
using System;
using System.Linq;
using UnityEngine;

class YourGameScenario : MonoBehaviour
{
    #region Attributes

    Vector3[] m_FoundPath;

    #endregion

    #region Unity events

    private void Start()
    {
        Nav3DManager.OnNav3DInit += () =>
        {
            Vector3 a = new Vector3(-10, -10, -10);
            Vector3 b = new Vector3(10, 10, 10);

            FindPath(a, b, PrintPath);
        };
    }

    private void OnDrawGizmos()
    {
        if (!Application.isPlaying || !enabled)
            return;

        if (m_FoundPath != null && m_FoundPath.Any())
        {
            for (int i = 0; i < m_FoundPath.Length - 1; i++)
                Gizmos.DrawLine(m_FoundPath[i], m_FoundPath[i + 1]);
        }
    }

    #endregion

    #region Service methods

    //your method for pathfinding from A to B
    void FindPath(Vector3 _A, Vector3 _B, Action<Vector3[]> _OnPathFound)
    {
        //create Path instance
        Path path = Nav3DPathfindingManager.PrefetchPath(_A, _B);
    }

```

```

//perform pathfinding from _A to _B
path.UpdatePath(_OnSuccess: () =>
{
    //notify that path was found
    Debug.Log("Path successfully found!");

    //invoke callback
    _OnPathFound(path.Trajectory);

    //finish work with Path instance
    path.Dispose();
});
}

void PrintPath(Vector3[] _Path)
{
    m_FoundPath = _Path;

    Debug.Log($"Path points: {string.Join(", ", m_FoundPath)}");
}

#endregion
}

```

Agents

Nav3DAgent is the main class that moves game units in the game scene space.

Nav3DAgent public methods can only be used after Nav3D has been initialized. Read more in “Nav3D initialization” section.

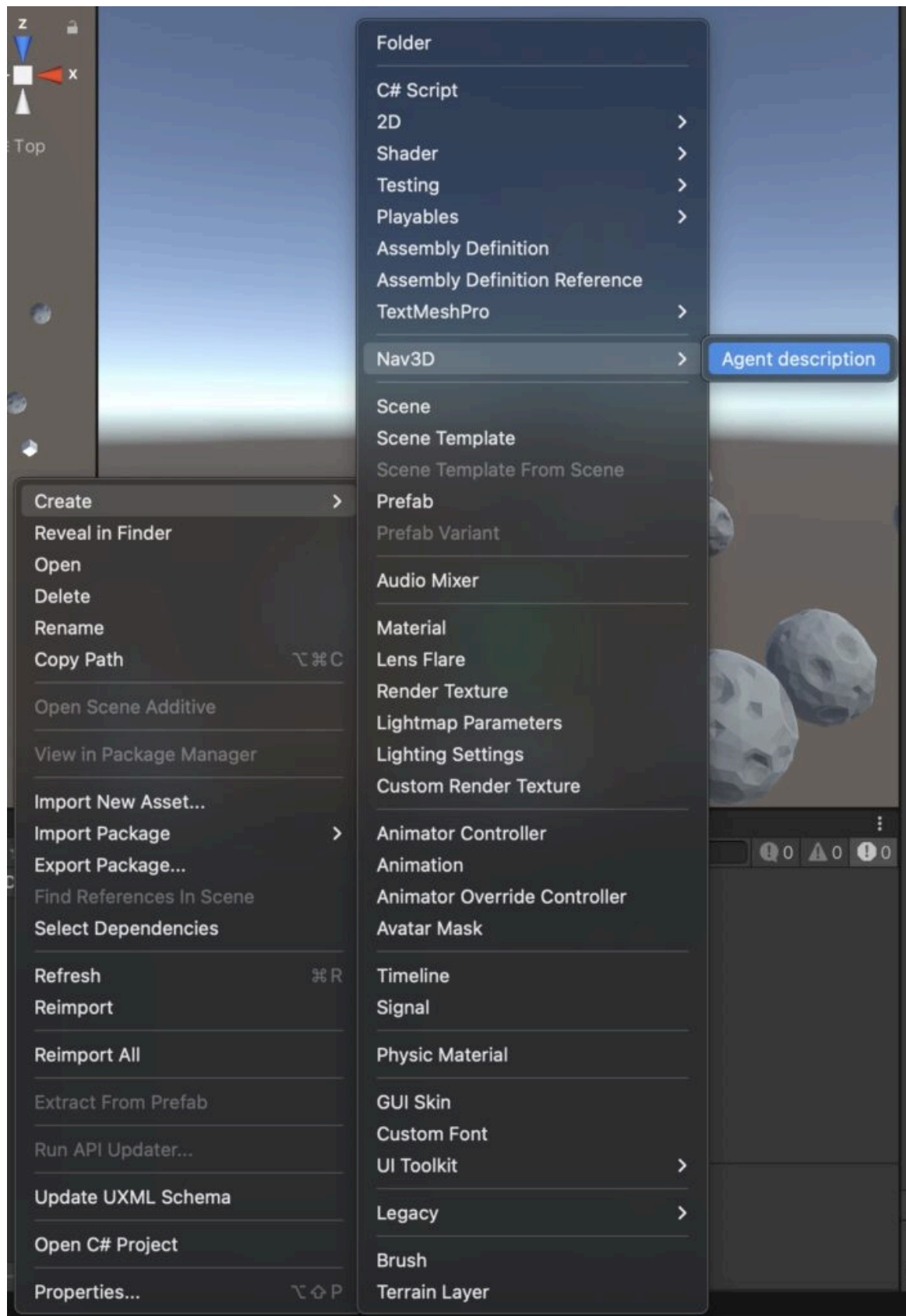
You can use this class as a component for your game units, or you can create it's inheritor.

To use the Nav3DAgent, you must configure its behavior. This can be done using the Nav3DAgentDescription class.

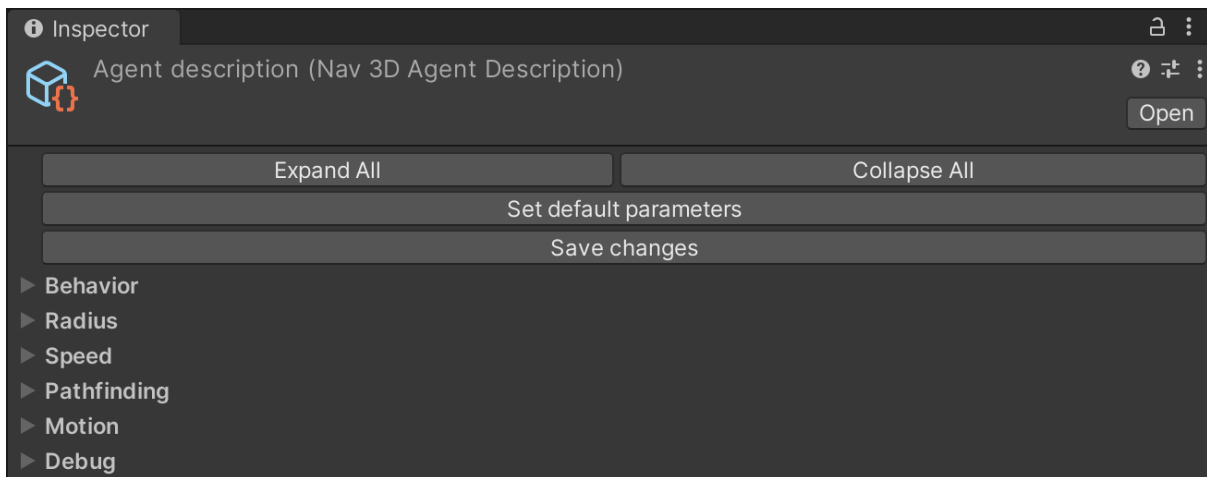
Nav3DAgentDescription

Nav3DAgentDescription serves as a description of the agent and allows you to customize its behavior and all necessary parameters.

This class inherits from ScriptableObject, so you can create an instance of it through the context menu of the Project tab.



After creating a description, in its inspector you will see the following list of parameter groups:



The Set default parameters button will speed up the configuration of all parameters by setting their default values. Here is an explanation of the purpose of the parameters and their allowable values.

Option	Property name in Nav3DAgentDescription	Type	Description	Value range
Behavior ▼ Behavior Behavior Type DEFAULT Motion Navigation Type GLOBAL_AND_LOCAL				
Behavior Type	BehaviorType	enum BehaviorType	Describes the type of agent behavior. - The agent can behave in a standard way (DEFAULT), i.e. search for a path, then follow it, or follow any other user instructions. The agent's behavior will be completely determined by its description. - The agent may have yielding behavior (YIELDING). It will not be able to execute user commands and will not have its own tasks. (An exception will be thrown if you try to call methods to move to the target.) His only concern will be to give way to agents nearby. It will interact with them using the Local avoidance mechanism, in accordance with its speed and radius parameters in description. - Also, the agent can be indifferent	DEFAULT, YIELDING, INDIFFERENT

			(INDIFFERENT). The agent will not do anything, it will just be there. Other agents will be able to bypass it when performing local avoidance. An attempt to call methods to follow to a target with this type of behavior will result in an exception being thrown, similar to YIELDING.	
Motion Navigation Type	MotionNavigationType	enum MotionNavigationType	The main purpose of all agents is to move towards some goal, be it another agent or a point in space. Agents can move using three ways of orientation in space. 1. Using the global path (GLOBAL). In this case, the agent will search for a path and will move along it, regardless of external conditions (other agents or edges of obstacles that are too close). 2. Using only local avoidance (LOCAL). The agent will not use pathfinding for movement, but will move towards the target in a straight line, dodging other agents and obstacles along the way when they meet on the course of movement. Of course, not every obstacle can be overcome in this way. 3. Finally, the agent can move towards the target using pathfinding and local avoidance at the same time (GLOBAL_AND_LOCAL). The agent will move along the found path and dodge other agents that come close, as well as obstacles that are too close.	GLOBAL, GLOBAL_AND_LOCAL, LOCAL

Radius

▼ Radius

Select radius value type:

Radius

Radius 	Radius	float	The agent's radius is used to perform local avoidance maneuvers. Establishing the correct radius is important for efficient local avoidance by both the agent and other agents nearby. We advise you to adjust the radius so that all the visual content of the game unit, which is the agent, is inside the sphere of the given radius.	>0
---	--------	-------	---	----

Speed

▼ Speed

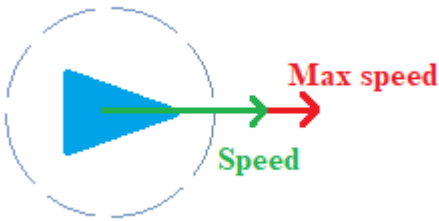
Select speed value type:

Speed

Select how to obtain max speed value:

Speed To Max Speed Multiplier

Max speed: 0.0115



Speed	Speed	float	The agent's speed. You can change it at any time if needed.	>0
Max Speed	MaxSpeed	float	The agent's maximum speed. This parameter is needed to perform local avoidance. In some situations, the agent may need to accelerate to avoid a collision. If the value is set too high, the agent may sometimes move in jerks when performing a local avoidance. So we advise you to set the maximum speed a little more than Speed (less than one and a half times more). For your convenience, the inspector has two ways to set this value. With a specific value. Or using a multiplier for Speed. We recommend using a multiplier so that the maximum speed depends on the Speed of an agent. Then MaxSpeed will change following the change in Speed anytime.	>Speed

Local avoidance

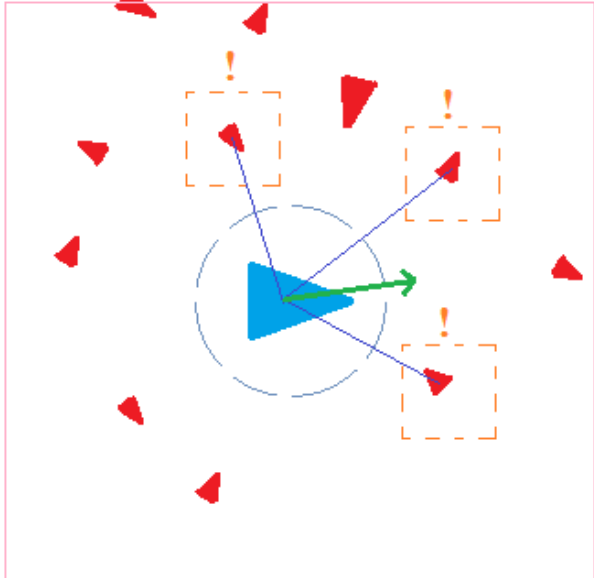
▼ Local avoidance

ORCA Tau

Agents considered number limit ☒

Agents number:

ORCA Tau	ORCATau	float	The time range for calculating the velocity obstacle (VO). We do not recommend changing this parameter. To better understand its meaning, the original article about ORCA: Reciprocal n-body	>0
----------	---------	-------	--	----

			Collision Avoidance will help you. Jur van den Berg, Stephen J. Guy, Ming Lin, and Dinesh Manocha https://gamma.cs.unc.edu/ORCA/publications/ORCA.pdf	
Agents considered number limit	UseConsideredAgentsNumberLimit	bool	Agent's area of perception  Enables a limit on the number of nearby agents considered for local avoidance. Enabling this option makes sense if you use a large number of agents in a tight space. In such a situation, each agent may have many neighbors, and performance may not be sufficient to provide high fps if all nearby agents are taken into account in the calculations.	true, false
Agents number	ConsideredAgentsNumberLimit	int	The number of agents that are taken into consideration from all nearby agents.	≥ 1

Pathfinding

▼ Pathfinding

Pathfinding timeout (ms)

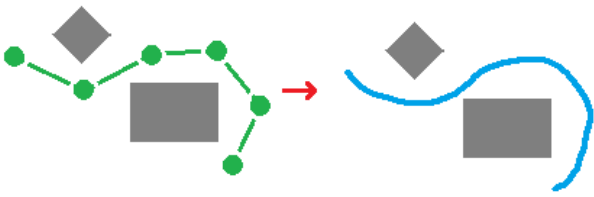
Smooth the path
☒

Samples per min bucket volume

Auto-update path on stagnant behavior
☒

Auto-update cooldown (ms)

Pathfinding timeout (ms)	Pathfinding Timeout	int	The maximum time allowed to perform pathfinding. If this time is exceeded, the pathfinding will be canceled and the <code>_OnPathfindingFail</code> callback will be called.	> 0
--------------------------	---------------------	-----	--	-------

Smooth the path	SmoothPath	bool	 Whether it is necessary to smooth the found path.	true, false
Samples per min bucket volume	SmoothRatio	int	The number of path segments per minimum octree cell size resulting from smoothing the original path. You should not excessively increase this parameter, as this increases the total pathfinding time.	>0
Auto-update path on stagnant behavior	AutoUpdatePath	bool	When using the local avoidance mechanism in conjunction with the global path search, a situation may arise when the path passes through a region of space where many other agents are present. Then our agent can move away from the found path, trying to avoid collisions with other agents. This process can continue for as long as you like, since trying to return to the original trajectory, the agent will always move away from it again, meeting other agents. Enable this option to autocorrect the path. Then, if the agent has moved away from his path far enough and is at a distance long enough, the path will be found again from the current position of the agent.	true, false
Auto-update cooldown (ms)	PathAutoUpdateCooldown	int	Minimum path auto-update period. If the area of space is too full of other agents, then path recalculation may be too frequent, which will negatively affect performance in case of a large number of agents in the scene. We recommend limiting the frequency of autocorrection to a few seconds.	>=1

Motion

▼ Motion

Target reach distance

0

Max rotation in degrees per fixed update tick

5

Target reach distance	TargetReachDistance	float	Distance to reach the target. When you order an agent to reach a target, the target will be considered reached when the agent's pivot matches the target's coordinate (the distance between them will be 0). This can be inconvenient if the target is a meshed object, say a planet in space. In this case, set this parameter equal to the sum of the radius of the	>=0
-----------------------	---------------------	-------	--	-----

			visible part of the agent and the radius of the planet. Then the agent will stop when it reaches the surface of the planet.	
Max rotation in degrees per fixed update tick	MaxAgentDegreesRotationPerTick	float	The maximum number of degrees an agent can turn when performing a turn in the direction of travel. In the process of movement, the agent turns towards the velocity vector. If the velocity vector changes abruptly in the process of movement, then the agent's turning may look abrupt. To make it smoother, set this parameter not too high.	≥ 0

Velocity blending

▼ *Velocity blending

Set default blending weights



Here you need to set the weights that will be used to blend the velocity vectors in different situations.

Agent follows global path, and there are both other agents and obstacles near.

Path Velocity Weight

1

Agents Avoidance Velocity Weight

10

Obstacle Avoidance Velocity Weight

5

Agent follows global path, and there are only obstacles near.

Path Velocity Weight 1

1

Obstacle Avoidance Velocity Weight 1

3

Agent follows global path, and there are only other agents near.

Path Velocity Weight 2

1

Agents Avoidance Velocity Weight 1

5

Agent perform only local avoidance, and there are both other agents and obstacles near.

Agents Avoidance Velocity Weight 2

1

Obstacle Avoidance Velocity Weight 2

1

When using global pathfinding and local avoidance, the agent moves using three velocity vectors:

- The velocity vector along the path.
- The velocity vector of avoidance from nearest agents.
- Obstacle avoidance velocity vector.

The agent blends these three velocities in different proportions and uses the resulting vector for movement. In order to tell the agent in what proportion to mix the speeds, we have introduced weights, the values of which you can adjust.

Not all three velocities can be blended in all situations. For example, there may be no obstacles near the agent, then only the velocity of following the path and the velocity of the nearest agents avoidance will be mixed.

So you can tell the agent how significant each velocity should be when getting the resulting one. If following a path is preferred, then the first weight can be made to be much larger than the last two. If it is strongly necessary to perform local avoidance maneuvers, then the weight of the velocity along the path should be made much smaller than the weight of the avoidance velocities.

Below are the explanations for the weight parameters. Note that separate parameters are used for each situation.

All weights are float and must be greater than 0.

The corresponding property names in Nav3DAgentDescription are:

PathVelocityWeight

PathVelocityWeight1

PathVelocityWeight2

AgentsAvoidanceVelocityWeight

AgentsAvoidanceVelocityWeight1

AgentsAvoidanceVelocityWeight2

ObstacleAvoidanceVelocityWeight

ObstacleAvoidanceVelocityWeight1

ObstacleAvoidanceVelocityWeight2

Debug

▼ Debug

Use Log



Log records count:

50

Use Log	UseLog	bool	Whether it is necessary to log agent work processes.	true, false
Log records count	LogSize	int	Agent log size.	>0

Randomly generated parameters

The Speed and Radius parameters can be set to a randomly generated value. This can be useful for game scenarios where there are many agents of the same type (for example, many similar birds). In order for their characteristics (radius and speed) to differ slightly, you can choose to generate a random value. You can then specify ranges for that value as well as its distribution.

There are two types of distribution, it is continuous uniform and Gaussian. The first one does the same as UnityEngine.Random.Range(a, b). The type of Gaussian distribution can be described as follows: most of the values will be chosen approximately from the middle of the range, but there will be few values that are closer to the ends of the range.

Below is an example of setting the radius for the range [0.1, 0.3] according to the Gaussian distribution.

▼ Radius

Select radius value type:

Specific value

Random ranged value

Select random distribution mode:

Default

Gaussian

Radius [Min, Max]:

0.1

0.3

Creating and configuring an agent description from code

All Nav3DAgentDescription parameters configured in the description inspector can also be configured from your code.

To do this, you need to create a variable of type Nav3DAgentDescription, then apply it to the agent. The most convenient way to create a description is to refer to the Nav3DAgentDescription.DefaultDescription property.

After setting the desired parameters in the code, you need to set the customized description to the agent. If your description contains parameters whose value is configured to be randomly generated, then before setting the description, the agent needs to get a variant of the description with generated parameters. This can be done through the Nav3DAgentDescription.GetDescriptionVariant() method. We recommend that you always use the GetDescriptionVariant() method to set the description for an agent via code.

The following is an example of creating a description and then applying it to an agent.

```
void ConfigureAgentDescription()
{
    Nav3DAgent myAgent = GetComponent<Nav3DAgent>();

    //create Nav3DAgentDescription instance with default parameters
    Nav3DAgentDescription myDescription = Nav3DAgentDescription.DefaultDescription;

    //set the parameters you want
    myDescription.Radius = 1.2f;
    myDescription.MotionNavigationType = MotionNavigationType.LOCAL;

    //apply description to agent
    myAgent.SetDescription(myDescription.GetDescriptionVariant());
}
```

Nav3DAgent

As mentioned above, you can use the Nav3DAgent script as a component for your game object and access it via GetComponent().

The following public methods have been implemented to operate with Nav3DAgent.

- public void SetDescription(Nav3DAgentDescription _Description)
Applies the description for your agent.
- public void MoveTo(Vector3 _Point, Action _OnReach = null, Action<PathfindingError> _OnPathfindingFail = null)
The order to move to the point. Any active order will be interrupted, the agent will start moving to the point, in accordance with the parameters of its description.
- public void MoveToEnqueue(Vector3 _Point, Action _OnReach = null, Action<PathfindingError> _OnPathfindingFail = null)

Add a destination to the queue. In this way, any number of sequence points can be added. If the agent executes the FollowTarget order, then this order is interrupted and it starts following the point.

- `public void FollowTarget(
 Transform _Target,
 float _OffsetToleranceUpdate,
 float _DistToReach = 0,
 Action _OnReach = null,
 Action _OnCancel = null,
 Action<PathfindingError> _OnPathfindingFail = null
)`

An order to chase a moving Transform.

Parameter values:

-Transform _Target - Transform to target.

-float _OffsetToleranceUpdate - the distance by which the shift of the target will cause the path to the target to be updated.

If your agent has the Motion Navigation Type of the description set to GLOBAL, or GLOBAL_AND_LOCAL, then each time the pursued Transform is offset by more than _OffsetToleranceUpdate, it will cause the path to be recalculated.

-float _DistToReach - distance to the target, approaching which the target will be considered reached. We would like to note that in the case when the agent has to look for a global path, the goal may never be reached, since having reached the end of the path it may turn out that the goal has moved over the past time. Thus, the condition of coincidence of the coordinates of the pursued target and the agent can be difficult to achieve.

-Action _OnReach - A delegate to be executed when the target is successfully reached.

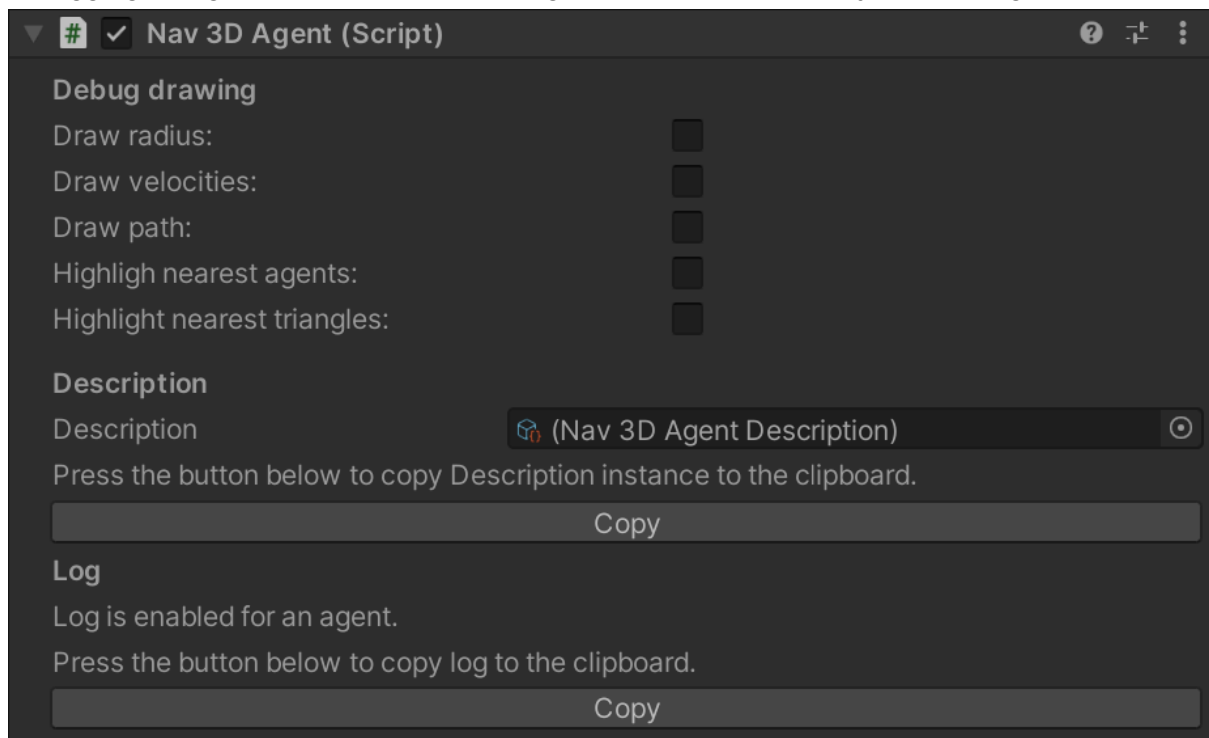
-Action _OnCancel - a delegate executed if the reference to the pursued Transform becomes null or missing.

- `public void Stop()`
Interrupts the execution of the current command.
- `public List<Nav3DAgent> GetAgentsInRadius(float _Radius,
 Predicate<Nav3DAgent> _Predicate = null)`
Allows you to get a list of agents that are within the _Radius radius from the agent. Calling this method every time when Update is firing can have a negative impact on performance, so it is recommended that you do not call it too often (for example, several times per second should be enough for any game scenarios.).
- `public void Visualize(bool _DrawRadius = true, bool _DrawPath = true, bool
 _DrawVelocities = true)`
Visualizes the agent velocities, radius and paths using Gizmos. May be useful for debugging in the Editor.

You can find examples of how to use Nav3DAgent in the demo scenes located in the Demo folder of the asset.

Nav3DAgent : Debug

The Nav3DAgent has an inspector that provides several useful possibilities to help with debugging and give a better understanding of the interaction of objects in the game scene.



Debug drawing

In this section, you can visualize the agent and its nearest environment. This should be done with Gizmos enabled on the game scene:

1. Visualization of the agent's radius.



2. Visualization of normalized agent velocities.

Yellow- the path following velocity vector (the direction vector from the agent's position to the next target point on the path).

Green- local avoidance vector.

Blue- the blended motion vector (a mixture of the path following vector and the local avoidance vector).



3. Visualization of the path the agent follows.



4. Visualization of nearby agents that can be taken into account when performing local avoidance.

When performing local avoidance, depending on the setting of the allowed number of agents under consideration (in the agent description), the n closest agents will be taken from all nearby agents.



5. Visualization of nearby obstacle triangles that are taken into account when performing local avoidance at the moment.



6. All together.



Agent log

Here you can copy the contents of the agent log to the clipboard by clicking the button if logging is enabled in the agent description.